

Cours intégré 2

Programmation concurrente

Des concepts au code : C++20 et OpenMP

Cours intégré – en alternance avec les TP « thread » (parties 1 et 2)

Au programme

les concepts du premier cours, dans un vrai langage

1. Créer et gérer des threads

`std::thread`, `join`, `detach`, `std::jthread`

2. Protéger les sections critiques

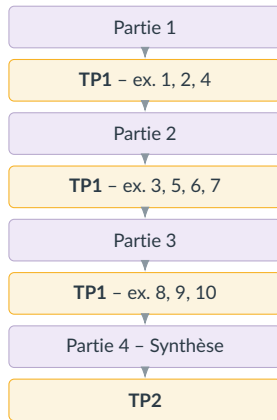
`std::mutex`, `RAll`, sémaphores

3. Deux outils complémentaires

`std::atomic`, `std::condition_variable`

4. Paralléliser avec OpenMP

avantages et coûts cachés



Du pseudo-code au C++20

vous connaissez déjà tous les concepts

Concept (C11)	Pseudo-code (TD)	C++20
Thread	–	<code>std::thread</code> , <code>std::jthread</code>
Mutex	<code>m.lock()</code> / <code>m.unlock()</code>	<code>std::mutex</code> + <code>std::lock_guard</code>
Sémaphore	<code>s.P()</code> / <code>s.V()</code>	<code>s.acquire()</code> / <code>s.release()</code>
Opération atomique	–	<code>std::atomic<int></code>
Attendre une condition	sémaphore « réveil » (philosophes, sol. 4)	<code>std::condition_variable</code>

Projet Qt Creator (.pro)

```
CONFIG += console c++20
LIBS    += -pthread
```

Exemples du cours

Tous les extraits de code du cours existent en version complète et compilable, avec un modèle de projet Qt Creator.

Partie 1

Créer et gérer des threads

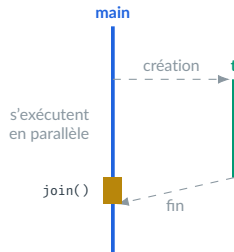
Création, cycle de vie et identification

Un premier thread

```
#include <thread>
```

```
void guichet()
{
    std::cout << "Guichet ouvert\n";
}

int main()
{
    std::thread t(guichet); // démarre
    std::cout << "main continue\n";
    t.join();               // attend t
    return 0;
}
```

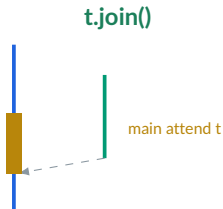


À retenir

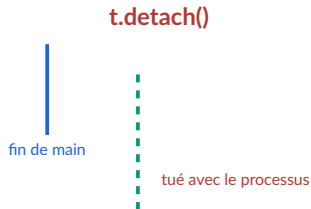
On passe au constructeur de `std::thread` la fonction à exécuter. Le thread démarre **immédiatement**; `join()` bloque l'appelant jusqu'à ce qu'il se termine.

join ou detach ?

que devient un thread quand on ne l'attend plus ?



L'appelant **attend** la fin du thread. C'est le cas normal.



Le thread n'est **plus associé à l'objet**, il vit sa vie / **on ne l'attend jamais**. Il est interrompu quand le programme se termine.

À retenir

Avant la destruction d'un objet `std::thread`, il faut **obligatoirement** avoir appelé `join()` ou `detach()`. Sinon, le programme s'arrête brutalement.

En pratique : `join()` presque toujours. `detach()` rend le thread incontrôlable.

Passer des arguments

par défaut, ils sont copiés

```
void ouvrir(std::string nom, int numero);
void compter_vente(int & compteur);

// copies
std::thread t1(ouvrir, "A", 1);

int ventes = 0;
// ne compile pas
std::thread t2(compter_vente, ventes);
// référence : OK
std::thread t3(compter_vente,
               std::ref(ventes));
```

- Les arguments suivent la fonction, dans l'ordre.
- `std::thread` les **copie** dans le nouveau thread.
- Pour transmettre une **référence**, il faut le dire explicitement : `std::ref(x)`, dans `<functional>`.

Rappel du C11

Avec `std::ref`, la variable locale `ventes` devient **partagée** entre les threads.

Identifier les threads et la machine

trois fonctions utiles

```
// identifiant du thread courant
std::this_thread::get_id()

// identifiant d'un objet thread
t.get_id()

// nombre de coeurs (indicatif, peut valoir 0)
std::thread::hardware_concurrency()

// endormir le thread courant
std::this_thread::sleep_for(
    std::chrono::milliseconds(500));
```

Pourquoi connaître le nombre de cœurs ?

Au-delà du nombre de cœurs, ajouter des threads de calcul ne va **pas plus vite** : ils se partagent les mêmes processeurs.

Piège

`sleep_for` sert à **simuler** un traitement long, **JAMAIS** à « synchroniser » des threads.

Lancer plusieurs threads

un vecteur de threads, et une boucle de `join`

```
void guichet(int numero) { ... }

std::vector<std::thread> guichets;

for (int i = 0; i < 4; ++i)
    guichets.emplace_back(guichet, i);

for (std::thread & t : guichets)
    t.join();
```

- ▶ `emplace_back` construit le thread **directement** dans le vecteur, avec les mêmes arguments que le constructeur.
- ▶ On lance **tous** les threads, **puis** on les attend tous.

Piège

`join()` dans la boucle de création : chaque thread attend la fin du précédent. Le programme est correct... mais **séquentiel**.

std::jthread (C++20)

un thread qui se joint tout seul

std::thread

```
{
    std::thread t(guichet, 1);
    // ...
    t.join();    // à ne pas oublier
}
```

std::jthread

```
{
    std::jthread t(guichet, 1);
    // ...
}    // join() automatique ici
```

À retenir

std::jthread appelle join() **automatiquement** dans son destructeur, à la sortie du bloc : impossible d'oublier. C'est le principe du **RAII**, qu'on reverra juste après avec les verrous.

Dans la vraie vie : préférez std::jthread. En TP, on commence volontairement par std::thread pour se familiariser avec les concepts.

À vous – sujet de TP

Partie 1 – exercices 1, 2 et 4

Créer, attendre et identifier des threads

Utilisez `std::thread`.

Exercice 2 : avant de corriger avec `join()`, prenez le temps d'expliquer le message d'erreur.

Exercice 4 : relancez plusieurs fois, et expliquez chaque affichage obtenu.

Partie 2

Protéger les sections critiques

mutex, RAI et sémaphores

La billetterie, en vrai

la course critique du premier cours, cette fois observable

```
int restantes = 1000000; // partagées
int vendues   = 0;

void guichet()
{
    while (restantes > 0) {
        restantes = restantes - 1;
        vendues   = vendues + 1;
    }
}
// main : 4 guichets, puis join
```

Trois exécutions successives

```
vendues = 1000000
vendues = 359892
vendues = 1000000
```

Piège

Le résultat change d'une exécution à l'autre : l'invariant `vendues + restantes = capacité` est brisé.

```
std::mutex  
#include <mutex>
```

```
std::mutex mut;    // partagé  
  
bool vendre()  
{  
    mut.lock();  
  
    bool ok = restantes > 0;    section critique  
    if (ok) {  
        restantes = restantes - 1;  
        vendues   = vendues + 1;  
    }  
  
    mut.unlock();  
    return ok;  
}
```

- ▶ `lock()` : prend le verrou ou attend passivement.
- ▶ `unlock()` : le rend.
- ▶ C'est exactement le patron du premier cours : `P()` avant, `V()` après.

Le piège du `unlock()` oublié

un chemin de sortie suffit

```
bool vendre()
{
    mut.lock();
    if (restantes == 0)
        return false; // verrouillé !
    restantes = restantes - 1;
    vendues   = vendues + 1;
    mut.unlock();
    return true;
}
```

Conséquence

Dès que le concert est complet, le premier guichet sort sans rendre le verrou : **tous les autres attendent pour toujours.**

Même problème avec une **exception** levée entre `lock()` et `unlock()`.

À retenir

En C++, il vaut mieux éviter d'appeler `lock()` / `unlock()` à la main. On confie le verrou à un **objet** qui le rendra à coup sûr.

Le RAI et `std::lock_guard`

Resource Acquisition Is Initialization

```
bool vendre()
{
    std::lock_guard<std::mutex> verrou(mut);           // lock() auto ici
    if (restantes == 0)
        return false;                                // unlock() auto
    restantes = restantes - 1;
    vendues   = vendues + 1;
    return true;                                       // unlock() auto
}
```

À retenir

RAII : une ressource est prise dans le **constructeur** d'un objet local, et rendue dans son **destructeur**. Le C++ garantit que le destructeur est appelé à la sortie du bloc : `return`, fin normale ou exception.

`std::unique_lock` un `lock_guard` plus souple

```
void vendre_encaisser()
{
    std::unique_lock<std::mutex> verrou(mut);
    bool ok = restantes > 0;
    if (ok) {
        restantes = restantes - 1;
        vendues   = vendues + 1;
    }
    verrou.unlock();    // fin de la SC

    if (ok)
        encaisser();
}
```

- ▶ Même garantie RAII que `lock_guard`.
- ▶ On peut en plus `unlock()` puis `lock()` à nouveau en cours de route.
- ▶ Indispensable avec `std::condition_variable` (partie 3).

Par défaut : `lock_guard`. Si besoin de souplesse : `unique_lock`.

Les sémaphores en C++20

`#include <semaphore>`

```
// 3 terminaux de paiement : 3 jetons
std::counting_semaphore<3> terminaux(3);

void encaisser()
{
    terminaux.acquire();    // P()
    // ... paiement ...
    terminaux.release();    // V()
}

std::binary_semaphore signal(0);
```

Cl1	C++20
-----	-------

s.P()	s.acquire()
s.V()	s.release()

Entre chevrons : le nombre **maximal** de jetons.
Entre parenthèses : le nombre **initial**.

Piège

Pas de RAI pour les sémaphores :
chaque `acquire()` doit avoir son
`release()`, sur tous les chemins.

Quel outil pour quel besoin ?

partie 2 en une diapo

Besoin	Outil	Concept du premier cours
Protéger une section critique	<code>std::mutex + std::lock_guard</code>	exclusion mutuelle
... en relâchant en cours de route	<code>std::mutex + std::unique_lock</code>	taille de la section critique
Limiter l'accès à N exemplaires	<code>std::counting_semaphore</code>	sémaphore compteur
Attendre un signal ponctuel	<code>std::binary_semaphore (init 0)</code>	sémaphore initialisé à 0

À retenir

Réflexe : **éviter** les `lock()` / `unlock()` manuels. Un verrou = un objet RAII.

À vous – sujet de TP

Partie 1 – exercices 3, 5, 6 et 7

Sections critiques, verrous et RAI

Exercice 3 : pour chaque exécution, identifiez quel thread affiche quoi.

Exercices 6 et 7 : avant de corriger, reliez le résultat observé à la course critique du premier cours. Et vérifiez que votre section critique n'est ni trop étroite, ni trop large.

Partie 3

Deux nouveaux outils

Opérations atomiques et attente d'une condition

```
std::atomic
#include <atomic>
```

```
std::atomic<int> visites(0);

void consulter()
{
    ++visites;    // atomique
}
```

Les opérations sur un `std::atomic` (`++`, `+=`, lecture, écriture) sont **atomiques** : les autres threads voient l'état avant ou après, jamais pendant.

À retenir

`std::atomic` : pour **une variable simple** modifiée par une seule opération (compteur, drapeau). Dès qu'un invariant lie **plusieurs** instructions ou variables : mutex.

Une opération, pas une séquence

```
std::atomic<int> restantes(100);
if (restantes > 0)    // atomique
    --restantes;    // atomique
```

Chaque instruction est atomique, mais pas le couple « tester, puis agir » : **la survente reste possible.**

Attendre qu'une condition devienne vraie

le problème que les verrous ne résolvent pas

Le concert est complet. Un acheteur veut attendre qu'une place soit **remise en vente**.

```
// Attente active : à proscrire
while (true) {
    std::lock_guard<std::mutex> v(mut);
    if (restantes > 0) {
        restantes = restantes - 1;
        break;
    }
}
```

Piège

Correct, mais le thread **tourne en boucle** : attente active, un cœur occupé à 100 % pour rien.

Déjà vu en TD

Dans la 4^e solution des philosophes, un philosophe affamé **s'endort** et ses voisins le **réveillent** quand ils ont fini de manger. Il fallait gérer ce réveil à la main, avec un sémaphore par philosophe.

À retenir

On veut : « endors-moi **jusqu'à ce que** cette condition soit vraie, et réveille-moi quand quelqu'un la change ».

std::condition_variable

```
#include <condition_variable>
```

L'état partagé

```
std::mutex mut;  
std::condition_variable place_libre;
```

Côté acheteur : attendre

```
std::unique_lock<std::mutex> verrou(mut);  
place_libre.wait(verrou,  
                 [&] { return restantes > 0; });  
restantes = restantes - 1;
```

Côté annulation : signaler

```
{  
    std::lock_guard<std::mutex> verrou(mut);  
    restantes = restantes + 1;  
}  
place_libre.notify_one();
```

Ce que fait `wait(verrou, condition)`

1. condition **déjà vraie** : rend la main aussitôt;
2. sinon, **relâche** le mutex et **endort** le thread;
3. à chaque réveil, **reprend** le mutex et **re-teste** la condition; ne rend la main que si elle est vraie.

La syntaxe entre crochets

`[&] { return restantes > 0; }` est une **fonction anonyme (lambda)**. Les crochets lui donnent accès aux variables environnantes. Ici, elle sert uniquement à *décrire ce qu'on attend*.

Deux règles à ne jamais oublier

avec une variable de condition

Attendre sans condition

```
place_libre.wait(verrou);    // faux  
restantes = restantes - 1;
```

Être réveillé ne garantit pas que la condition est vraie : un autre acheteur a pu prendre la place entre-temps, et la norme autorise des réveils « parasites », sans aucune notification.

→ toujours donner la condition à `wait`.

Modifier l'état hors du mutex

```
restantes = restantes + 1;    // faux  
place_libre.notify_one();
```

La condition porte sur une donnée partagée : sa modification est une **section critique** comme une autre.

→ modifier sous le mutex, **puis** notifier.

À retenir

Pour attendre : `wait(verrou, [&]{ condition });` Pour signaler : modifier sous verrou, **puis** notify.

À vous – sujet de TP

Partie 1 – exercices 8, 9 et 10

Atomicité, attente et philosophes

Exercice 9 : l'affichage doit attendre que l'initialisation soit terminée. Quel outil permet d'attendre sans boucler ?

Exercice 10 : avant de coder, rappelez-vous quelle condition de Coffman casse la solution choisie.

Partie 4

Paralléliser avec OpenMP

Avantages et coûts cachés

OpenMP : paralléliser par annotations

un standard pour le calcul parallèle en C, C++ et Fortran

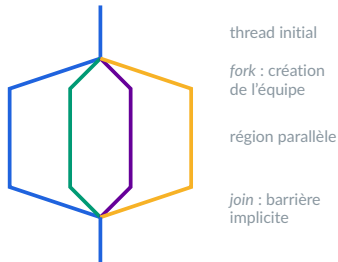
- ▶ On écrit un code **séquentiel**.
- ▶ On **annotate** les parties à paralléliser avec des directives `#pragma omp`.
- ▶ Le compilateur transforme le code annoté ; la bibliothèque OpenMP **crée les threads**, leur répartit le travail et les joint.

Projet Qt Creator (.pro)

```
QMAKE_CXXFLAGS += -fopenmp
LIBS += -fopenmp
```

et `#include <omp.h>` dans le code

Modèle *fork-join*



Vocabulaire

Fork-join est le terme de la norme OpenMP. Il s'agit bien de **threads**, qui partagent la mémoire : aucun rapport avec l'appel `fork()` vu en programmation système, qui crée un **processus** avec sa propre mémoire.

parallel for

la directive la plus utilisée

Avec des threads (TP2, exercice 2)

```
void remplir(std::vector<double> & prix, int debut, int fin);

int tranche = N / nbThreads;
std::vector<std::thread> threads;

for (int t = 0; t < nbThreads; ++t)
    threads.emplace_back(remplir, std::ref(prix),
                        t * tranche, (t+1) * tranche);
for (std::thread & th : threads)
    th.join();
```

Avec OpenMP

```
#pragma omp parallel for
for (int i = 0; i < N; ++i)
    prix[i] = 20.0 + (i % 50);
```



indices répartis entre les threads

À retenir

OpenMP découpe les itérations en tranches, lance un thread par tranche, puis les joint. Condition indispensable : les itérations doivent être **indépendantes** les unes des autres.

Variables partagées : les courses critiques n'ont pas disparu

reduction et critical

```
double total = 0.0;
#pragma omp parallel for
for (int i = 0; i < N; ++i)
    total += prix[i]; // course !
```

Par défaut, les variables déclarées **avant** la boucle sont **partagées**.

reduction : une copie par thread, fusionnées à la fin

```
#pragma omp parallel for reduction(+:total)
for (int i = 0; i < N; ++i)
    total += prix[i];
```

critical : l'équivalent d'un mutex

```
#pragma omp critical
total += prix[i];
```

À retenir

OpenMP crée les threads pour vous, mais **ne protège rien** à votre place. Tous les concepts vus jusqu'ici s'appliquent : ressources partagées, sections critiques, invariants.

Les coûts cachés du parallélisme

plus de threads ne veut pas toujours dire plus vite

Surcoût de création

Créer, répartir et joindre des threads prend du temps. Sur une petite boucle, la version parallèle peut être **plus lente**.

Sections critiques

Un `critical` ou un mutex dans la boucle fait passer les threads **un par un** : section critique trop large.

Plus de threads que de cœurs

Au-delà de `hardware_concurrency()`, les threads de calcul se partagent les mêmes cœurs.

Loi d'Amdahl

Si une fraction p du programme est parallélisable, le gain sur n cœurs est au plus

$$\frac{1}{(1 - p) + p/n}$$

partie parallélisable	gain sur 8 cœurs	gain max.
50 %	×1,8	×2
90 %	×4,7	×10
95 %	×5,9	×20

Ce qu'une boucle parallèle ne sait pas faire

et qu'il faut savoir reconnaître

Des itérations dépendantes

```
#pragma omp parallel for
for (int i = 1; i < N; ++i)
    cumul[i] = cumul[i-1] + prix[i];
```

L'itération i a besoin du résultat de $i-1$, qui est peut-être calculé par un autre thread... ou pas encore.

S'arrêter en cours de route

Une boucle `parallel for` doit parcourir **toutes** ses itérations : pas de `break`, pas de `return` à l'intérieur.

Et l'ordre dans lequel les itérations sont exécutées **n'est pas garanti**.

À retenir

Avant d'ajouter `#pragma omp parallel for`, demandez-vous : les itérations sont-elles indépendantes, et le résultat dépend-il de l'ordre de parcours ?

Partie 5

Synthèse

Ce qu'il faut retenir du cours

Des concepts aux outils

les deux cours en une diapo

Concept (C11)	Outil C++20	Outil OpenMP
Thread	<code>std::jthread</code> , <code>std::thread</code>	<code>parallel</code> , <code>parallel for</code>
Section critique	<code>std::mutex</code> + <code>lock_guard</code>	<code>critical</code>
Opération atomique	<code>std::atomic</code>	<code>reduction</code> (pour l'accumulation)
Sémaphore compteur	<code>std::counting_semaphore</code>	-
Attendre une condition	<code>std::condition_variable</code>	-

Les réflexes

Un verrou = un objet RAII.

Un compteur simple = `std::atomic`.

Une attente = `while` + `wait`.

Ce qui ne change pas

Les outils changent, les problèmes restent :
courses critiques, interblocage et famine
s'anticipent comme au premier cours.

À vous – sujet de TP

Partie 2

Threads et OpenMP sur des cas concrets

Exercice 1 : un thread par client, sur le serveur echo.

Exercices 2 à 4 : la même boucle avec des threads, puis avec OpenMP, puis ses limites.

Exercice 5 : comparez les temps, et gardez en tête les coûts cachés.